

Programação Recursiva: processamento de listas em Lisp — Parte2

Orlando de A Figueiredo

IGCE/Unesp

21 de Agosto de 2017

Parte 2

Mais exercícios, com pequenas diferenças.

- ▶ Funções com duas listas como parâmetros
- ▶ Acesso a outros elementos além do primeiro
- ▶ Recursão com base em listas e números

juntar (isto é, *append*)

Dicas:

- ▶ Basta fazer a recursão em uma das listas.
- ▶ O caso base não retorna lista vazia.

```
(juntar '(4 5) '(10 15))  
>>> '(4 5 10 15)  
(juntar '() '(10 10 5))  
>>> '(10 10 5)
```

Mais dicas: juntar

- ▶ A última expressão corresponde à recursão da cauda, já pronta.
- ▶ O que falta para se tornar à resposta esperada? (que está na expressão anterior.)
- ▶ (l-um e l-dois continuam em outros exemplos à frente.)

```
(define l-um '(3 1 2 3))  
(define l-dois '(10 3 1 3))  
(juntar l-um l-dois)  
>>> '(3 1 2 3 10 3 1 3)  
(juntar (cdr l-um) l-dois)  
>>> '(1 2 3 10 3 1 3)
```

substituir

Dicas:

- ▶ O exemplo mostra símbolos, mas poderiam ser números. Use sempre `equal?` pois contempla todos os casos.

```
(substituir 'a 'd '(c b a b))  
>>> '(c b d b)
```

gaguinho (ou *stammer*)

Dicas:

- ▶ O elemento deve ser inserido duas vezes, ou seja, cons deve aparecer duas vezes.

```
(gaguinho '(3 7 8 b))  
>>> '(3 3 7 7 8 8 b b)
```

Mais dicas: gaguinho

Dicas:

- O que falta à recursão da cauda para se tornar a resposta?

```
(gaguinho l-um)
>>> (3 3 1 1 2 2 3 3)
(gaguinho (cdr l-um))
>>> '(1 1 2 2 3 3)
```

alternar

Dicas:

- ▶ Troca de posição entre elementos sucessivos.
- ▶ Se houver um número ímpar de elementos, o elemento final fica inalterado.
- ▶ É preciso visitar o primeiro e o segundo elementos de uma só vez.
- ▶ É preciso chamar a recursão removendo dois elementos (duas chamadas de `cdr`).
- ▶ É preciso testar, além da lista nula, a lista com um único elemento (`(null? (cdr l))`).

```
(alternar '(3 9 1 8 0))  
>>> '(9 3 8 1 0)
```

Mais dicas: alternar

```
(alternar (cdr (cdr l-um)))  
>>> (3 2)  
(car (l-um))  
>>> 3  
(car (cdr l-um))  
>>> 1  
(alternar (l-um))  
>>> (1 3 3 2)
```

ímpares

Dicas:

- ▶ Ficam os elementos das posições ímpares.
- ▶ (E não os elementos cujos valores são ímpares. Isso já era feito com `filtrar!`)
- ▶ Esse exercício é muito parecido com `alternar`, mas mais simples!

```
(ímpares '(3 3 2 1 0))  
>>> '(3 2 0)
```

ordenado?

Dicas:

- ▶ Permanece a ideia de acessar o primeiro e o segundo elementos.
- ▶ Mas a recursão deve ser feita de um em um elemento.
- ▶ Ainda assim, é preciso testar o caso da lista unitária (por quê?).

```
(ordenado? '(1 10 100))  
>>> #t  
(ordenado? '(1 10 9 15))  
>>> #f
```

segue

Dicas:

- ▶ Dado um valor que aparece uma ou mais vezes numa lista, retornar os elementos que o seguem nas diversas ocorrências.
- ▶ É preciso acessar o segundo elemento em certas condições.

```
(segue 'a '(b a c a d a b d))  
>>> '(c d b)  
(segue 'b (a b c b d b))  
>>> '(c d ())
```

zíper

Dicas:

- ▶ Duas listas, com recursão simultânea.
- ▶ A sobra da maior é anexada no final.

```
(zíper '(a 3 z) '(9 c b))  
>>> '(a 9 3 c z b)  
(zíper '(1 10) '(-1 0 -2 7))  
>>> '(1 -1 10 0 -2 7)
```

misturar

Dicas:

- ▶ Duas listas ordenadas, mas apenas uma é consumida de cada vez.

```
(misturar '(1 20 50) '(2 8 30))  
>>> '(1 2 8 20 30 50)
```

pegar (ou *take*)

Dicas:

- ▶ Há dois parâmetros: o número e a lista.
- ▶ O número é decrementado (*sub1*) quando a lista perde um elemento (na recursão).

```
(pegar 3 '(1 0 12 15 8))  
>>> '(1 0 12)
```

tirar (ou *drop*)

Dicas:

- ▶ É o dual de pegar

```
(tirar 3 '(1 0 12 15 8))  
>>> '(15 8)
```